Problem Statement

Objective

The goal of this assignment is to develop a custom Named Entity Recognition (NER) model to identify and extract key elements from recipe text, such as ingredients, quantities, and units of measurement. By applying this model, you will gain a deeper understanding of how to convert unstructured recipe text into structured data, thereby enhancing recipe search engines, cooking assistants, and meal planning applications.

Business Value:

The business objective is to leverage the increasing popularity of online cooking platforms and meal planning apps by enhancing user experience. This can be achieved by implementing a custom Named Entity Recognition (NER) model to automatically tag ingredients, quantities, and recipe names. This automation will streamline the process of organizing recipes, improve searchability, and enable users to easily find recipes based on available ingredients, portion sizes, or specific dietary requirements. This will ultimately reduce the labor-intensive and inefficient manual tagging process, providing a more accessible and efficient way for businesses in the food and recipe industry to manage their recipe databases.

Dataset Overview

Context

This dataset comprises culinary recipes with a focus on ingredient extraction and analysis. Each recipe features a structured ingredient list with labeled components, identifying ingredients, quantities, and units. This diverse collection supports tasks like recipe understanding and culinary knowledge discovery, enabling the development of models for information extraction in the culinary domain.

Content

The dataset consists of a single JSON file, ingredient_and_quantity.json, which contains a collection of culinary recipes. Each recipe is represented as a JSON object with the following keys:

- **input**: A raw ingredient list from a recipe, presented as a space-separated string.
- **pos**: A space-separated string containing corresponding part-of-speech (POS) tags or Named Entity Recognition (NER) labels, identifying quantities, ingredients, and units for each token in the input string.

Acknowledgements

This dataset, focused on culinary recipes, was provided by UpGrad for educational and research purposes.

Scoring and Penalty

- **Total Marks**: **170** (100 for code notebook, 70 for report)
- **Extension and penalty**: As given in your learner handbooks

Instructions

1. Each learner should attempt this assignment individually.
2. Programming Language: Python
3. You will be provided with the data set and a starter notebook. You have to perform analyses in the starter notebook only.
4. You must not change any headings, subheadings, questions or tasks in your notebook as this can cause problems with grading.
5. For analyses and processing tasks, you should use only the following libraries: Json, Pandas, Matplotlib, Seaborn, Sklearn_crfsuite and SpaCy.
6. The data will have inconsistencies and outliers.
7. You are encouraged to search the web and consult AI tools for conceptual understanding. However, using plagiarized code is strictly prohibited and using purely AI-generated code is strongly discouraged.
8. Submitting plagiarized and purely AI-generated code will result in significant penalties to your scores.

Submission Guidelines

1. You are required to upload the following two files as your solution:
    - An ***interactive Python notebook (.ipynb)*** that contains your code. Use the following naming convention:
      'Identifying_Key_Entities_in_Recipe_Data_<your_name>.ipynb'
    - A ***report document (.pdf)*** that presents your visualisations, analysis, results, insights and outcomes. Use the following naming convention:
      'Identifying_Key_Entities_in_Recipe_Data_<your_name>.pdf'
2. These files should only be generated from the starter files which are provided to you.
3. The submitted Jupyter notebook and report should contain your name and the assignment title.
4. Mention all assumptions, if made, in the report.
5. The report should include the overall approach to the assignment, covering the problem statement, methodology, techniques used and key insights.

Results Expected from Learners

In the starter notebook, you will find headings, subheadings, and checkpoints stating the tasks you need to perform. The marks associated with each checkpoint will also be mentioned in the notebook. Keep in mind not to edit the cells with marking schemes and questions. You can find a brief description of the tasks below.

1. Data Description [0 marks]

(a) The project uses ingredient_and_quantity.json for training a CRF model to extract ingredients, quantities, and units from recipes.

Install necessary libraries:

!pip install sklearn-crfsuite==0.5.0 spacy==3.5.3

!python -m spacy download en_core_web_sm

2. Data Ingestion and Preparation [25 marks]

(a) Read Recipe Data and Preparation [12 marks]

i. Define a load_json_to_dataframe function to read data from a JSON file. [7 marks]

ii. Execute the load_json_to_dataframe function. [2 marks]
iii. Describe the DataFrame, print first five rows, dimensions, and information. [3 marks]

(b) Recipe Data Manipulation [13 marks]

i. Create input_tokens and pos_tokens columns by splitting data. [3 marks]
ii. Validate length equality of input_tokens and pos_tokens. [2 marks]
iii. Define and execute a unique_labels function. [2 marks]
iv. Provide insights and identify indexes requiring cleaning. [1 mark]
v. Drop rows with invalid data. [2 marks]
vi. Update and validate input_length and pos_length. [3 marks]

3. Train Validation Split (70 train - 30 val) [6 marks]

(a) Splitting the Dataset [6 marks]

i. Split the dataset into training and validation sets with a 70:30 ratio. [1 mark]
ii. Print the first five rows of both training and validation DataFrames. [1 mark]
iii. Extract data into X_train, X_val, y_train, and y_val. [2 marks]
iv. Display the number of unique labels present in y_train. [2 marks]

4. Exploratory Recipe Data Analysis on Training Dataset [16 marks]

(a) Data flattening and Token Extraction [4 marks]

i. Define a flatten_list function and initialize the dataset name. [2 marks]
ii. Define and execute an extract_and_validate_tokens function. [2 marks]

(b) Token Categorization and Frequency Analysis [5 marks]

i. Define and execute a categorize_tokens function. [2 marks]
ii. Define and execute a get_top_frequent_items function for ingredients and units. [3 marks]

(c) Visualization and Insights [7 marks]

i. Define a plot_top_items function. [2 marks]
ii. Plot bar graphs for top ingredients and units in the training dataset. [2 marks]
iii. Provide insights based on the visualization. [3 marks]

5. Exploratory Recipe Data Analysis on Validation Dataset (Optional) [0 marks]

(a) Execute EDA on Validation Dataset with Insights (Optional) [0 marks]

i. Initialize the dataset name as "Validation."
ii. Utilize the functions defined in the previous section and perform EDA on the validation dataset.
iii. Generate bar plots for the top 10 ingredients and units in the validation dataset.
iv. Provide insights based on the visualization of the validation samples, comparing the findings with the training dataset analysis.

6. Feature Extraction For CRF Model [30 marks]

(a) Define Feature Functions [10 marks]

i. Define keywords for unit, quantity, and cooking methods. Create a pattern to recognize numerical values. [3 marks]

ii. Define a word2features function to extract features for each token, incorporating core, quantity/unit detection, and contextual features. [7 marks]

(b) Recipe Level Features [2 marks]

Define a sent2features function to apply word2features to all tokens in a recipe. [2 marks]

(c) Convert Data to Feature Sets [6 marks]

i. Create X_train_features and X_val_features using sent2features on training and validation data. [2 marks]

ii. Create y_train_labels and y_val_labels by converting y_train and y_val to lists. [2 marks]

iii. Print the lengths of the training and validation feature sets and labels to confirm data consistency. [2 marks]

(d) Applying Weights to Feature Sets [12 marks]

i. Flatten y_train_labels to count individual labels. [2 marks]

ii. Count label frequencies and calculate total samples. [2 marks]

iii. Compute a weight_dict using the inverse frequency method, penalizing the 'ingredient' label. [2 marks]

iv. Define an extract_features_with_class_weights function to apply weights during feature extraction. [4 marks]

v. Create X_train_weighted_features and X_val_weighted_features by applying weights to the feature sets. [2 marks]

7. Model Building and Training [10 marks]

(a) CRF Model Initialization and Training [5 marks]

i. Initialize a CRF model using sklearn_crfsuite.CRF with specified hyperparameters. [3 marks]

ii. Set the class_weight attribute of the CRF model to the weight_dict calculated in the previous section. [1 mark]

iii. Train the CRF model using the weighted training data. [1 mark]

(b) Training Dataset Evaluation [4 marks]

i. Predict labels for the training dataset using the trained CRF model. [1 mark]

ii. Generate a flat classification report using flat_classification_report to evaluate the model's performance on the training dataset. [2 marks]

iii. Create and print a confusion matrix to visualize the model's predictions on the training dataset. [1 mark]

(c) Save the CRF Model [1 marks]

i. Save the trained CRF model [1 marks]

8. Prediction and Model Evaluation [3 marks]

(a) Validation Set Prediction and Evaluation [3 marks]

i. Predict labels for the validation dataset using the trained CRF model. [1 mark]

ii. Generate a flat classification report using flat_classification_report to evaluate the model's performance on the validation dataset. [1 marks]

iii. Create and print a confusion matrix to visualize the model's predictions on the validation dataset. [1 marks]

9. Error Analysis on Validation Data [10 marks]

(a) Error Investigation [8 marks]

i. Flatten Labels, Initialize Error Data [2 marks]

ii. Iterate and Collect Error Information [2 marks]

iii. Create Error DataFrame, Print Accuracy [1 mark]

iv. Analyze Errors by Label Type [3 marks]

(b) Validation Insights [2 marks]

i. Provide Insights from Error Analysis [2 marks]

10. Conclusion (Optional) [0 marks]

(a) Concluding Insights [0 marks]

i. Briefly summarize the project's key findings and insights.

ii. Comment on the CRF model's overall performance.

iii. Mention any limitations or challenges encountered.

Evaluation Guidelines

The following rubrics will be used while judging your solutions to the above tasks.

Table 1: Rubrics

Criteria	Meets expectations	Does not meet expectations
Data Ingestion and Preparation	1. All data quality issues are correctly identified and reported. 2. The meanings of the variables are correctly interpreted and written either in the comments or text.	1. Data quality issues are overlooked or are not identified correctly such as missing values, outliers and other data quality issues. 2. The variables are interpreted incorrectly or the meaning of variables is not mentioned.

Train Validation Split	1. The dataset is split into training and validation sets with a 70:30 ratio. 2. The first five rows of both training and validation DataFrames are printed. 3. The number of unique labels present in y_train is displayed.	1. The dataset is not split correctly. 2. The first five rows of both training and validation DataFrames are not printed. 3. The number of unique labels present in y_train is not displayed.

Criteria	Meets expectations	Does not meet expectations
Exploratory Recipe Data Analysis on Training Dataset	1. The analysis addresses the right problem and follows a clear structure. 2. Important driver variables are identified. 3. Business-driven, type-driven, and data-driven metrics are created and utilized. 4. Multivariate analysis is performed correctly. 5. Insights are explained correctly in the comments. 6. Appropriate plots are created and labeled correctly.	1. The analysis does not address the right problem or lacks structure. 2. Important driver variables are not identified 3. New metrics are not derived or explained. 4. Derived metrics are not analysed correctly/are insufficiently utilised. 5. Insights are not mentioned. 6. Relevant plots are not created or labeled correctly.
Feature Extraction For CRF Model	1. Keywords for unit, quantity, and cooking methods are defined. 2. A pattern to recognize numerical values is created. 3. A word2features function is defined to extract features for each token, incorporating core, quantity/unit detection, and contextual features.	1. Keywords for unit, quantity, and cooking methods are not defined. 2. A pattern to recognize numerical values is not created. 3. A word2features function is not defined or does not incorporate the required features.

Model Building and Training	1. A CRF model is initialized using sklearn_crfsuite.CRF with specified hyperparameters. 2. The class_weight attribute of the CRF model is set to the weight_dict calculated in the previous section. 3. The CRF model is trained using the weighted training data.	1. A CRF model is not initialized correctly. 2. The class_weight attribute is not set correctly. 3. The CRF model is not trained using the weighted training data.
Prediction and Model Evaluation	1. Labels for the validation dataset are predicted using the trained CRF model. 2. A flat classification report is generated to evaluate the model's performance on the validation dataset. 3. A confusion matrix is created and printed to visualize the model's predictions on the validation dataset.	1. Labels for the validation dataset are not predicted correctly. 2. A flat classification report is not generated. 3. A confusion matrix is not created or printed.
Error Analysis on Validation Data	1. Errors are investigated by flattening labels, initializing error data, iterating and collecting error information, creating an error DataFrame, and printing accuracy. 2. Errors are analyzed by label type. 3. Insights from error analysis are provided.	1. Errors are not investigated correctly. An error DataFrame is not created. 2. Errors are not analyzed by label type. 3. Insights from error analysis are not provided.
Report	1. The report has a clear structure, is not lengthy and explains the process and results concisely in simple language. 2. The report includes visualisations and insights derived from them. 3. Assumptions made, if any, are	1. The report lacks structure, is lengthy or does not emphasise important observations. The language used is complicated for business people to understand. 2. The report is missing visualisations or fails to provide meaningful

	stated clearly.	insights. 3. Assumptions made, if any, are not stated clearly.
Conciseness and readability of the code	1. The code is concise and syntactically correct. Wherever appropriate, built-in functions and standard libraries are used instead of writing long code (if-else statements, loops, etc.). 2. Custom functions are used to perform repetitive tasks. 3. The code is readable with appropriately named variables and detailed comments are written wherever necessary.	1. Long and complex code is used instead of shorter built-in functions. 2. Custom functions are not used to perform repetitive tasks resulting in the same piece of code being repeated multiple times. 3. Code readability is poor because of vaguely named variables or lack of comments wherever necessary.